

Day3-03 学习文档 v1.0 : Runtime Architecture —— Conversation & Message Data Model

本文是《从零实现 Agent Runtime》学习阶段的 Day3-03 正式学习文档。

Day3-01 讨论 Runtime 的架构设计原则，Day3-02 讨论 Runtime 的第一版核心组件。Day3-03 继续往下走，重点讨论 Runtime 内部最重要的数据模型：Conversation 与 Message。

与上一节的联系

Day3-02 解决的问题是：

如果要实现 mini-agent-runtime 的第一版，Runtime 内部到底有哪些核心组件？

当时我们得到的核心对象是：

- Runtime
- Conversation
- Message
- ContextBuilder
- LLMClient
- ToolRegistry
- Tool

Day3-03 进一步追问：

Runtime 里面最重要的数据，到底应该长什么样？

这一节的核心结论是：

Conversation 保存的不是普通聊天记录，而是 Runtime 运行过程中发生过的事实事件。

也就是说，Message 不只是 user / assistant 两种聊天消息。对于 Agent Runtime 来说，Tool Call、Tool Result、Approval、Error、Summary、Checkpoint 等都可能成为 Message 或 Event。

目录

为什么 Conversation 和 Message 是 Runtime 的核心？

一次 Agent Loop 到底发生了什么？

Prompt、Plan 与 Runtime Event 的区别

Tool Call 为什么不是 Runtime 硬拆出来的？

LLM 如何知道有哪些 Tool？

没有 Tool 时，Conversation 会记录什么？

Message 是聊天记录还是领域事件？

Conversation 本质上更像 Event Log

Message 的生命周期

Message 的类型设计：Role vs Type

Internal Message vs Provider Message

Immutable Message：为什么 Message 应尽量不可变

Conversation 为什么不只是 Message[]

Conversation 的职责边界

Conversation 的未来演化：Checkpoint、Branch、Replay

第一版数据模型设计

架构设计方法总结

Production Thinking：从 Demo 到工业级

下一节学习计划

写书 TODO

写书素材

1. 为什么 Conversation 和 Message 是 Runtime 的核心？

很多人第一次学习 Agent Runtime 时，会把注意力放在：

- Prompt；
- Tool Call；
- Memory；
- MCP；
- Workflow；
- 模型 API。

但如果从 Runtime 作者的视角看，真正更底层的问题是：

Runtime 到底在记录什么？

几乎所有高级能力最后都会回到 Message：

Summary	依赖历史 Message
Memory	从 Message 中提取长期信息
Streaming	可能产生 Delta Message
Workflow	需要记录阶段和状态变化
Checkpoint	需要保存某一时刻的运行历史
Replay	需要重新读取事件历史
Human Approval	需要记录请求和审批结果
Trace	需要记录执行路径
Telemetry	需要记录运行事件

所以，Conversation 和 Message 不是附属对象，而是 Runtime 的地基。

如果 Message 设计得太简单，后面的 Summary、Memory、Replay、Human Approval 都会变得别扭；如果 Conversation 只是一个随手维护的数组，后续想做持久化、分支、回放、恢复，就会很快遇到边界问题。

2. 一次 Agent Loop 到底发生了什么？

假设用户输入：

帮我查询北京今天的天气，然后告诉我是否适合出去跑步。

从用户视角看，这似乎就是一个完整 Prompt，然后模型给出一个完整回答。

但从 Runtime 作者视角看，内部可能发生了更多事情：

```

User Input
|
v
Conversation.append(UserMessage)
|
v
ContextBuilder.build(conversation)
|
v
LLM receives Context + Tool Schemas
|
v
LLM returns ToolCall(weather)
|
v
Conversation.append(AssistantToolCallMessage)
|
v
Runtime executes weather tool
|
v
Conversation.append(ToolResultMessage)
|
v
ContextBuilder.build(conversation)
|
v
LLM returns final answer
|
v
Conversation.append(AssistantMessage)

```

这时 Conversation 里不应该只有：

```

UserMessage
AssistantMessage

```

更合理的是：

```

UserMessage
AssistantToolCallMessage
ToolResultMessage
AssistantMessage

```

关键变化在于：

Conversation 保存的是 Runtime 发生过的重要事实，而不是只保存最终展示给用户的聊天文本。

3. Prompt、Plan 与 Runtime Event 的区别

今天讨论中最容易混淆的是三个概念：

```

Prompt
Plan
Runtime Event

```

它们不是同一个东西。

Prompt

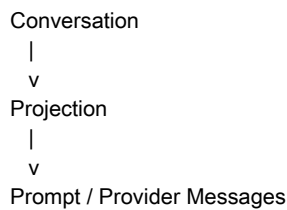
Prompt 是用户输入，或者 ContextBuilder 拼出来发给 LLM 的输入。

Prompt = 给 LLM 看的输入

它可能包含：

- 用户当前问题；
- 历史消息；
- System 指令；
- Tool Schema；
- Summary；
- Memory；
- 当前任务上下文。

Prompt 本身不是 Conversation 的核心存储对象。它更像是 ContextBuilder 根据 Conversation 临时生成的一种 Projection。



Plan

Plan 是面向未来的任务拆解。

例如 Cursor 的 Plan 模式，可能把一个目标拆成：

- Task 1: 分析代码结构
- Task 2: 定位 Bug
- Task 3: 修改实现
- Task 4: 运行测试

Plan 不是历史事实，它是未来准备要做的事情。

Plan = 未来要做什么

Planner 是高级能力，不是 Runtime 第一版必须存在的组件。

Runtime Event

Runtime Event 是已经发生过、值得记录的事实。

Runtime Event = 过去发生了什么

例如：

- 用户输入了一句话；
- LLM 返回了一个 ToolCall；
- Tool 执行成功；
- Tool 执行失败；
- 用户批准了高风险操作；
- Summary 被更新；
- Workflow 进入下一阶段。

所以三者的关系可以理解为：

Prompt 面向 LLM 输入
 Plan 面向未来动作
 Event 面向过去事实

不要把它们混成一个对象。

4. Tool Call 为什么不是 Runtime 硬拆出来的？

一个常见误解是：

Runtime 根据用户 Prompt 把过程拆成多个 Runtime Event。

更准确的说法是：

Runtime 不负责凭空拆分事实，它负责观察 Agent Loop 中真实发生的事情，并把有价值的事实追加到 Conversation 中。

例如天气查询场景里：

LLM returns ToolCall(weather)

这不是 Runtime 自己假装拆出来的，而是 LLM 在收到 Context 和 Tool Schema 后做出的决策。

Runtime 的职责是：

看见 LLM 返回了 ToolCall；

判断这个 ToolCall 是否有效；

找到对应 Tool；

执行 Tool；

把 ToolCall 和 ToolResult 作为事实写入 Conversation；

继续下一轮 Agent Loop。

因此，Runtime 不是事件的幻想制造者，而是事件的记录者和执行协调者。

5. LLM 如何知道有哪些 Tool？

LLM 本身并不知道当前 Runtime 注册了哪些工具。

真正知道 Tool 的是 Runtime。

完整流程应该是：

```
Conversation
|
v
ContextBuilder.build()
|
v
Runtime reads ToolRegistry
|
v
Runtime sends Context + Tool Schemas to LLM
|
v
LLM chooses whether to return ToolCall
|
v
Runtime validates and executes ToolCall
```

也就是说，LLM 能返回 ToolCall 的前提是：Runtime 在调用 LLM 前，把当前可用工具的 Schema 一起提供给了模型。

一个简化示意：

```
const context = contextBuilder.build(conversation)
const tools = toolRegistry.listSchemas()

const response = await llm.chat({
  messages: context.messages,
  tools,
})
```

这里要注意一件事：

Runtime 决定有哪些 Tool 可用；LLM 决定当前是否需要调用某个 Tool；Runtime 再决定是否允许和执行这个 ToolCall。

LLM 负责生成决策，Runtime 负责执行裁决。

6. 没有 Tool 时，Conversation 会记录什么？

如果一次 Agent Loop 中没有 Tool，Conversation 不应该凭空多出 ToolCall 或 ToolResult。

例如用户输入：

你好，请介绍一下 React。

可能发生的事件只有：

```
UserMessage
AssistantMessage
```

这说明：

发生几件值得记录的事，Conversation 就记录几件事。

不同场景下 Conversation 的形态不同：

Case 1: 纯聊天

```
UserMessage
AssistantMessage
```

Case 2: Tool Call

```
UserMessage
AssistantToolCallMessage
ToolResultMessage
AssistantMessage
```

Case 3: Human Approval

```
UserMessage
ApprovalRequestMessage
ApprovalResultMessage
ToolResultMessage
AssistantMessage
```

Runtime 不应该规定每轮必须有固定数量的 Message，而应该记录真实发生、可观察、需要保留的领域事件。

7. Message 是聊天记录还是领域事件？

在传统聊天系统里，Message 往往就是：

```
type ChatMessage = {
  role: "user" | "assistant" | "system"
  content: string
}
```

这适合普通聊天，但不适合完整的 Agent Runtime。

Agent Runtime 会发生很多不属于普通聊天的事件：

```
ToolCall
ToolResult
Error
Retry
Interrupt
ApprovalRequest
ApprovalResult
MemoryWrite
SummaryUpdate
Checkpoint
WorkflowStep
StreamingDelta
```

这些都不是用户和助手之间的自然语言聊天，却是 Runtime 运行历史中的重要事实。

因此，在 mini-agent-runtime 里，Message 应该更接近：

```
Domain Event
```

也就是说：

Message 是 Runtime 认为值得保留的领域事件的一种表现形式。

这样理解后，Message 不再只是 UI 展示对象，而是 Runtime 的事实记录单位。

8. Conversation 本质上更像 Event Log

如果 Message 是领域事件，那么 Conversation 就不只是聊天记录容器。

它更像：

```
Event Log
```

也就是一条事实日志。

```
Conversation
|
v
Event 1: UserMessage
Event 2: AssistantToolCallMessage
Event 3: ToolResultMessage
Event 4: AssistantMessage
Event 5: SummaryUpdatedMessage
Event 6: ApprovalRequestedMessage
```

这个视角非常重要，因为它会影响后面所有能力：

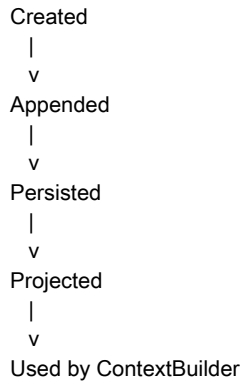
- Summary 不再只是聊天摘要，而是对事件历史的压缩；
- Memory 不再只是保存聊天偏好，也可以保存 Tool 使用历史和失败模式；
- Replay 不再是重放文本，而是重放事件；
- Checkpoint 不再是保存字符串数组，而是保存某个事件位置的运行状态；
- Debug 不再只看最终回答，而是看整个事件路径。

一句话：

Conversation 是 Runtime 世界的事实账本。

9. Message 的生命周期

一条 Runtime Message 通常会经历几个阶段。



Created

某件事情发生后，Runtime 创建一条 Message。

例如：

```
const message = RuntimeMessage.user({
  content: "帮我查询北京天气",
})
```

Appended

Message 被追加到 Conversation。

```
conversation.append(message)
```

Persisted

如果 Runtime 支持持久化，这条 Message 会被写入存储。

```
ConversationRepository.save(message)
```

Projected

ContextBuilder 会读取 Message，并把它投影为模型供应商需要的输入格式。

```
RuntimeMessage
|
v
OpenAIMessage / ClaudeMessage / GeminiMessage
```

Used

LLM 根据这些投影后的上下文生成下一步响应。

这个生命周期再次说明：

Message 是 Runtime 内部模型；Provider Message 是 API 调用时的外部格式。

10. Message 的类型设计：Role vs Type

很多入门教程会用 role 区分消息：

```
type Role = "system" | "user" | "assistant" | "tool"
```

但 Agent Runtime 里，仅靠 Role 很快不够。

例如：


```
assistant message
assistant tool call
assistant reasoning summary
assistant final answer
```

它们都可能来自 assistant，但语义完全不同。

再例如：

```
tool result success
tool result error
tool result timeout
```

它们都可能来自 tool，但 Runtime 需要区分不同类型。

所以更合理的方式是同时保留：

```
role = 谁产生的
type = 发生了什么
```

示例：

```
type MessageRole =
  | "system"
  | "user"
  | "assistant"
  | "tool"
  | "runtime"
  | "human"

type MessageType =
  | "user_message"
  | "assistant_message"
  | "assistant_tool_call"
  | "tool_result"
  | "tool_error"
  | "approval_request"
  | "approval_result"
  | "summary_update"
  | "memory_write"
  | "checkpoint"
```

这样设计后，Runtime 可以清晰表达：

```
谁产生了这条 Message？
这条 Message 表示什么事件？
```

11. Internal Message vs Provider Message

Day03-03 最重要的架构判断之一是：

Runtime 永远不要直接使用 OpenAI Message、Claude Message 或任何第三方 SDK Message 作为自己的核心数据模型。

第三方 API 的数据结构只是 API Contract，不是 Runtime 的 Domain Model。

如果内部直接保存 OpenAI 格式：

```
type OpenAIMessage = {
  role: "system" | "user" | "assistant" | "tool"
  content: string
}

class Conversation {
  messages: OpenAIMessage[]
}
```

短期看很快，长期会很痛。

当未来要支持 Claude、Gemini、OpenRouter、Azure OpenAI 时，Conversation 会被供应商格式绑死。

更好的结构是：

```
RuntimeMessage
|
+-- OpenAIAdapter --> OpenAI Message
|
+-- ClaudeAdapter --> Claude Message
|
+-- GeminiAdapter --> Gemini Message
```

也就是：

```
interface RuntimeMessage<TPayload = unknown> {
  id: string
  conversationId: string
  role: MessageRole
  type: MessageType
  payload: TPayload
  createdAt: Date
  metadata?: Record<string, unknown>
}
```

Provider Adapter 负责转换：

```
interface ProviderMessageAdapter<TProviderMessage> {
  toProviderMessages(messages: RuntimeMessage[]): TProviderMessage[]
}
```

架构原则是：

Internal Model 由业务决定；Provider Model 由供应商决定。两者之间必须有 Adapter。

12. Immutable Message：为什么 Message 应尽量不可变

Message 一旦追加到 Conversation，应该尽量不可变。

原因是 Conversation 保存的是事实历史。

如果历史 Message 可以随意修改，会带来几个问题：

- Debug 时无法确认当时到底发生了什么；
- Replay 时无法复现原始过程；
- Checkpoint 无法可靠恢复；
- Audit Log 失去可信度；
- Summary 和 Memory 可能基于已经被修改的历史。

因此第一版可以采用简单原则：

Message append-only

如果确实需要修正，优先追加一条新事件，而不是改旧事件。

例如：

```
ToolResultMessage
CorrectionMessage
```

而不是直接覆盖原来的 ToolResult。

这与 Event Sourcing 的思想非常接近：

不修改历史，只追加新的事实。

13. Conversation 为什么不只是 Message[]

第一版可以把 Conversation 写成：

```
class Conversation {  
    messages: RuntimeMessage[]  
}
```

这可以作为 Demo 起点，但它不是最终形态。

随着 Runtime 变复杂，Conversation 可能需要支持：

- Metadata ;
- Summary ;
- Checkpoint ;
- Branch ;
- Replay ;
- Fork ;
- Resume ;
- Time Travel ;
- Persistence Cursor ;
- Message Index ;
- Visibility Policy ;
- Retention Policy。

所以更准确地说：

Conversation = Message Log + Metadata + Evolution Mechanism

第一版我们仍然可以从 Message[] 开始，但心里要知道它只是最小实现，不是完整抽象。

14. Conversation 的职责边界

Conversation 最容易被写成上帝对象：

```
class Conversation {  
    messages: RuntimeMessage[]  
  
    async callLLM() {}  
    async callTool() {}  
    async summarize() {}  
    async saveMemory() {}  
    async stream() {}  
}
```

这种设计看起来方便，但会很快失控。

Conversation 的职责应该是：

拥有 Message

追加 Message
读取 Message
维护 Conversation Metadata

它不应该负责：

- 调用 LLM；
- 执行 Tool；
- 生成 Summary；
- 写入 Memory；
- 调度 Workflow；
- 决定 Agent Loop；
- 拼接 Provider Prompt。

一句话：

Conversation 只负责拥有消息，而不是处理消息。

可以用一个判断标准：

如果删掉某个能力，Conversation 还是不是 Conversation？

例如：

删掉 Summary：Conversation 还是 Conversation
删掉 LLM：Conversation 还是 Conversation
删掉 Tool：Conversation 还是 Conversation
删掉 Message：Conversation 就不存在了

所以 Message 属于 Conversation；Summary、LLM、Tool 不属于 Conversation。

15. Conversation 的未来演化：Checkpoint、Branch、Replay

第一阶段：

```
Conversation
|
v
Message[]
```

这是最小聊天 Runtime。

第二阶段会出现 Summary：

```
Conversation
|
+-- recent messages
+-- summary
```

第三阶段会出现 Checkpoint：

```
Conversation
|
+-- messages
+-- checkpoint #1
+-- checkpoint #2
+-- checkpoint #3
```

Checkpoint 解决的是长任务恢复问题。

第四阶段会出现 Branch：

```
Conversation
|
+-- Branch A
+-- Branch B
+-- Branch C
```

Branch 解决的是多路径探索问题。

第五阶段会出现 Replay :

```
Event Log
|
v
Replay
|
v
Rebuild State
```

Replay 解决的是调试、复现和恢复问题。

这时 Conversation 越来越像 Git Repository :

- 保存历史 ;
- 支持分支 ;
- 支持回滚 ;
- 支持重放 ;
- 支持对比。

所以未来的 Conversation 不只是聊天数组 , 而是 Runtime 的运行历史仓库。

16. 第一版数据模型设计

mini-agent-runtime 第一版可以先保持克制。

Conversation

```
interface Conversation {
  id: string
  title?: string
  metadata: ConversationMetadata
  messages: RuntimeMessage[]
}
```

Conversation 拥有 Message , 仅此而已。

RuntimeMessage

```
interface RuntimeMessage<TPayload = unknown> {
  id: string
  conversationId: string
  role: MessageRole
  type: MessageType
  payload: TPayload
  createdAt: Date
  metadata?: MessageMetadata
}
```

注意这里没有 OpenAI、Claude、Gemini。

Payload

不同 MessageType 对应不同 Payload。

```

interface UserMessagePayload {
    content: string
}

interface AssistantMessagePayload {
    content: string
}

interface ToolCallPayload {
    toolCallId: string
    toolName: string
    arguments: Record<string, unknown>
}

interface ToolResultPayload {
    toolCallId: string
    toolName: string
    result: unknown
}

interface ToolErrorPayload {
    toolCallId: string
    toolName: string
    error: {
        message: string
        code?: string
    }
}

```

Metadata

Conversation Metadata 可以先保持简单：

```

interface ConversationMetadata {
    createdAt: Date
    updatedAt: Date
    summary?: string
    tags?: string[]
}

```

Message Metadata 也可以保持开放：

```

type MessageMetadata = Record<string, unknown>

```

Provider Adapter

Provider Adapter 负责把 Runtime 内部消息转成模型 API 需要的格式。

```

interface MessageAdapter<TProviderMessage> {
    toProviderMessages(messages: RuntimeMessage[]): TProviderMessage[]
}

```

这样第一版结构就比较清晰：

```

graph TD
    Conversation --> v1[ ]
    v1 --> RuntimeMessage[RuntimeMessage[]]
    RuntimeMessage --> v2[ ]
    v2 --> ContextBuilder[ContextBuilder]
    ContextBuilder --> v3[ ]
    v3 --> ProviderAdapter[Provider Adapter]
    ProviderAdapter --> v4[ ]
    v4 --> OpenAI[OpenAI / Claude / Gemini]

```

17. 架构设计方法总结

今天真正学到的不是几个接口，而是一套设计 Runtime 的方法。

1. Domain First, Runtime Last

不要一开始就写：

```
class Runtime {  
  async run() {}  
}
```

应该先找 Runtime 世界里的领域对象：

```
Conversation  
Message  
ToolCall  
ToolResult  
Memory  
Checkpoint  
Workflow
```

让这些对象先成立，再写 Runtime 去协调它们。

2. Ownership 比 API 更重要

先问：

```
谁拥有谁？  
谁可以修改谁？  
谁只是读取谁？
```

例如：

```
Conversation owns Message  
ToolRegistry owns Tool  
MemoryStore owns Memory  
Runtime owns orchestration, not domain data
```

3. Dependency 指向稳定模块

依赖方向应该是：

```
Runtime  
|  
v  
Conversation
```

而不是：

```
Conversation  
|  
v  
Runtime
```

Conversation 是更稳定的领域对象；Runtime 是更容易变化的协调者。

4. Coordinator 允许有受控耦合

Runtime 天生会耦合多个模块：

```
Runtime  
|  
+-- Conversation  
+-- ContextBuilder  
+-- LLMClient  
+-- ToolRegistry
```

这是合理耦合，不是无序耦合。

好的架构不是没有耦合，而是让变化的影响面尽可能小。

5. Every Module Can Run Alone

一个模块如果离开 Runtime 就不能测试，往往说明它设计得太重。

例如：

```
const conversation = new Conversation()
conversation.append(userMessage)
expect(conversation.messages.length).toBe(1)
```

Conversation 不需要 LLM 也能测试。

ToolRegistry 不需要 Runtime 也能测试。

ContextBuilder 不需要真实模型也能测试。

Runtime 最后只是把这些可独立工作的模块组织起来。

18. Production Thinking : 从 Demo 到工业级

Demo 版 Conversation :

```
const messages = []
```

工业级 Conversation :

```
Conversation
|
+-- Message Log
+-- Metadata
+-- Summary
+-- Checkpoint
+-- Branch
+-- Replay
+-- Persistence
+-- Access Policy
```

Demo 版 Message :

```
type Message = {
  role: "user" | "assistant"
  content: string
}
```

工业级 Message :

```
RuntimeMessage
|
+-- role
+-- type
+-- payload
+-- metadata
+-- timestamp
+-- causality / parent relation
+-- visibility
```

Demo 版 Runtime :

```
input -> LLM -> output
```

工业级 Runtime :

```
input
```


- > append UserMessage
- > build Context
- > inject Tool Schemas
- > call LLM
- > append ToolCall
- > execute Tool
- > append ToolResult
- > rebuild Context
- > call LLM
- > append AssistantMessage
- > persist / trace / checkpoint

这就是从 API 调用走向 Runtime 设计的分界线。

19. 下一节学习计划

Day03-04 可以继续做：

Runtime Architecture Diagram

建议重点输出：

- Runtime 组件依赖图；
- Conversation / Message 数据流图；
- Agent Loop 时序图；
- Tool Call 执行时序图；
- Internal Message 到 Provider Message 的 Adapter 图；
- Day03 架构原则总览图。

Day04 则进入：

Context Builder

Day04 会回答：

同一个模型，为什么在 ChatGPT、Cursor、Claude Code 和 OpenAI Agents SDK 中表现得像不同的 AI？

核心原因不是模型本身，而是 Runtime 如何构建 Context。

20. 写书 TODO

- 补充一张 Prompt / Plan / Event 三层对比图。
- 补充一张 Conversation as Event Log 的图。
- 补充 Role vs Type 的代码示例。
- 补充 Internal Message -> Provider Message 的 Adapter 示例。
- 在 Day03 总结中加入 Domain First, Runtime Last。
- 在后续每一天增加两个固定栏目：
- Architect Thinking；
- Production Thinking。
- 在实现阶段为以下模块分别写单元测试：

- Conversation ;
- RuntimeMessage ;
- ToolRegistry ;
- ContextBuilder ;
- Provider Adapter。

21. 写书素材

可以保留为书中金句：

Conversation 保存的不是聊天，而是 Runtime 中发生过的事实事件。

Prompt 是 ContextBuilder 的投影结果，不是 Runtime 的核心存储对象。

Planner 拆分的是未来要做什么；Conversation 记录的是已经发生了什么。

Runtime 不负责凭空制造事件，它负责观察、执行和记录真实发生的事件。

Runtime 决定有哪些 Tool 可用；LLM 决定当前是否需要调用 Tool；Runtime 再决定是否允许和执行。

Internal Model 由业务决定；Provider Model 由供应商决定。

永远不要让第三方 SDK 的数据结构成为你的核心数据结构。

一个优秀 Runtime 的设计顺序不是先写 Runtime，而是先设计 Runtime 世界里的领域对象。

好的架构不是没有耦合，而是让变化的影响面尽可能小。

一个模块如果离开 Runtime 就不能测试，说明这个模块设计失败了。

本节最终结论

Day03-03 的最终结论可以浓缩成一句话：

Agent Runtime 的核心不是 Prompt，也不是 Tool Call，而是 Conversation 如何保存 Runtime 运行过程中发生过的领域事件。

第一版 mini-agent-runtime 可以从 Conversation + RuntimeMessage[] 开始，但设计上要提前留出 Internal Message、Provider Adapter、Immutable Message、Checkpoint、Branch、Replay 的演化空间。